

Experiment No. – 11							
Date of Performance:	3rd Aug 2026						
Date of Submission:	4th Aug 2026						
Conceptual Understanding (02)	Execution (1.5)	Problem Solving & Debugging(02)	Professional Conduct & Lab Ethics(02)	Output/ Result Accuracy (1.5)	Documentation & Result Analysis (01)	Experiment Total (10)	Sign with Date

Experiment No. 11 Study of Swing

11.1 Aim: Implementation of Swing components: Write a Java program to implement Swing components namely Buttons, JLabels, Checkboxes, Radio Buttons, JScrollPane, JList, JComboBox, Trees, Tables Scroll pan Menus and Toolbars to design interactive GUI.

11.2 Course Outcome: Design graphical user interface (GUI) based applications using AWT and Swing components with event-driven programming.

11.3 Learning Objectives: To create graphical user interface using Swing components

11.4 Requirement: Java development kit (JDK as per OS) , any IDE NetBeans, eclipse

11.5 Related Theory:

Swing is a Java Foundation Classes [JFC] library and an extension of the Abstract Window Toolkit [AWT]. Java Swing offers much-improved functionality over AWT, new components, expanded components features, and excellent event handling. In Java, you can create swing components for building graphical user interfaces (GUIs). Swing is a part of Java's standard library and provides a rich set of components for creating desktop applications.

The javax.swing package provides classes for java swing API such as JButton, JTextField, JTextArea, JRadioButton, JCheckbox, JMenu, JColorChooser etc.

Java JButton: Used to create a labeled button which may perform designated task after clicking.

JButton constructors:-

JButton() : creates a button with no text and icon.

JButton(String s): creates a button with the specified text.

JButton(Icon i): creates a button with the specified icon object.

Table 13.1: JButton Methods

JButton Methods	Description
void setText(String s)	It is used to set specific text on button
String getText()	It is used to return the text of the button.
void setEnabled(boolean b)	It is used to enable or disable the button.
Icon getIcon()	It is used to get the Icon of the button.
void setMnemonic(int a)	It is used to set the mnemonic on the button.
void addActionListener(ActionListener a)	It is used to add the action listener to this object.

Example:

```
import java.awt.event.*;
import javax.swing.*;
public class ButtonExample
{
    public static void main(String[] args)
    {
        JFrame f=new JFrame("Button Example");
        final JTextField tf=new JTextField();
        tf.setBounds(50,50, 150,20);
        JButton b=new JButton("Click Here");
        b.setBounds(50,100,95,30);
        b.addActionListener(new ActionListener()
        {
            public void actionPerformed(ActionEvent e)
            {
                tf.setText("Welcome to Javatpoint.");
            }
        })
        f.add(b);f.add(tf);
        f.setSize(400,400);
        f.setLayout(null);
        f.setVisible(true);
    }
}
```

JLabel:

Objective of JLabel is to place read only text in a container.

JLabel constructors:-

JLabel(): Label instance and with an empty string for the title.

JLabel(String s): JLabel with the specified text.

JLabel(Icon i): JLabel with the specified image.

Table 13.2: JLabel Methods

JLabel Methods	Description
String getText()	t returns the text string that a label displays.
void setText(String text)	It defines the single line of text this component will display.
void setHorizontalAlignment(int alignment)	It sets the alignment of the label's contents along the X axis.
Icon getIcon()	It returns the graphic image that the label displays.
int getHorizontalAlignment()	It returns the alignment of the label's contents along the X axis.

Example:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class LabelExample extends Frame implements ActionListener
{
    JTextField tf; JLabel l; JButton b;
    LabelExample()
    {
        tf=new JTextField();
        tf.setBounds(50,50, 150,20);
        l=new JLabel();
        l.setBounds(50,100, 250,20);
        b=new JButton("Find IP");
        b.setBounds(50,150,95,30);
        b.addActionListener(this);
        add(b);add(tf);add(l);
        setSize(400,400);
        setLayout(null);
        setVisible(true);
    }
    public void actionPerformed(ActionEvent e)
    {
        try{
```

```

        String host=tf.getText();
        String ip=java.net.InetAddress.getByName(host).getHostAddress();
        l.setText("IP of "+host+" is: "+ip);
    } catch (Exception ex) {System.out.println(ex);}
    }
    public static void main(String[] args)
    {
        new LabelExample();
    }
}

```

JTextField:

JTextField allows the editing of a single line text.

JTextField Constructor:

JTextField():Creates a new TextField: Creates a new TextField initialized with the specified text.

JTextField(String text): Creates a new TextField initialized with the specified text and columns.

JTextField(int columns): Creates a new empty TextField with the specified number of columns.

Table 13.3: JTextField Methods

JTextField Methods	Description
void addActionListener(ActionListener l)	It is used to add the specified action listener to receive action events from this textfield.
Action getAction()	It returns the currently set Action for this ActionEvent source, or null if no Action is set.
void setFont(Font f)	It is used to set the current font.
void removeActionListener(ActionListener l)	It is used to remove the specified action listener so that it no longer receives action events from this textfield

Example:

```

import javax.swing.*;
import java.awt.event.*;
public class TextFieldExample implements ActionListener
{
    JTextField tf1,tf2,tf3;
    JButton b1,b2;
    TextFieldExample(){
        JFrame f= new JFrame();
        tf1=new JTextField();
        tf1.setBounds(50,50,150,20);
        tf2=new JTextField();
        tf2.setBounds(50,100,150,20);
        tf3=new JTextField();
        tf3.setBounds(50,150,150,20);
    }
}

```

```

        tf3.setEditable(false);
        b1=new JButton("+");
        b1.setBounds(50,200,50,50);
        b2=new JButton("-");
        b2.setBounds(120,200,50,50);
        b1.addActionListener(this);
        b2.addActionListener(this);
        f.add(tf1);f.add(tf2);f.add(tf3);f.add(b1);f.add(b2);
        f.setSize(300,300);
        f.setLayout(null);
        f.setVisible(true);
    }
    public void actionPerformed(ActionEvent e)
    {
        String s1=tf1.getText();
        String s2=tf2.getText();
        int a=Integer.parseInt(s1);
        int b=Integer.parseInt(s2);
        int c=0;
        if(e.getSource()==b1)
        {
            c=a+b;
        }else if(e.getSource()==b2)
        {
            c=a-b;
        }
        String result=String.valueOf(c);
        tf3.setText(result);
    }
    public static void main(String[] args)
    {
        new TextFieldExample();
    }
}

```

JTextArea: JTextArea class is a multi line region that displays text.

JTextArea Constructor:

JTextArea():Creates a text area that displays no text initially.

JTextArea(String s): Creates a text area that displays specified text initially.

JTextArea(int row, int column): Creates a text area with the specified number of rows and columns that displays no text initially.

JTextArea(String s, int row, int column): Creates a text area with the specified number of rows and columns that displays specific text.

Table 13.4: JTextArea Methods

JTextArea Methods	Description
void setRows(int rows)	It is used to set specified number of rows.
void setColumns(int cols)	It is used to set specified number of columns.

<code>void setFont(Font f)</code>	It is used to set the specified font.
<code>void insert(String s, int position)</code>	It is used to insert the specified text on the specified position.
<code>void append(String s)</code>	It is used to append the given text to the end of the document.

Example:

```
import javax.swing.*;
public class TextAreaExample
{
    TextAreaExample() {
        JFrame f= new JFrame();
        JTextArea area=new JTextArea("Welcome to javatpoint");
        area.setBounds(10,30, 200,200);
        f.add(area);
        f.setSize(300,300);
        f.setLayout(null);
        f.setVisible(true);
    }
    public static void main(String args[])
    {
        new TextAreaExample();
    }
}
```

JPasswordField: text component specialized for password entry.

JPasswordField Constructor:

`JPasswordField()`: Constructs a new `JPasswordField`, with a default document, null starting text string, and 0 column width.

`JPasswordField(int columns)`: Constructs a new empty `JPasswordField` with the specified number of columns.

`JPasswordField(String text)`: Constructs a new `JPasswordField` initialized with the specified text.

`JPasswordField(String text, int columns)`: Construct a new `JPasswordField` initialized with the specified text and columns.

Example:

```
import javax.swing.*;
public class PasswordFieldExample
{
    public static void main(String[] args) {
        JFrame f=new JFrame("Password Field Example");
        JPasswordField value = new JPasswordField();
        JLabel l1=new JLabel("Password:");
        l1.setBounds(20,100, 80,30);
        value.setBounds(100,100,100,30);
        f.add(value); f.add(l1);
        f.setSize(300,300);
        f.setLayout(null);
    }
}
```

```

        f.setVisible(true);
    }
}

```

JCheckBox: It is used to turn an option on or off.

JCheckBox Constructor:

JCheckBox(): Creates an initially unselected check box button with no text, no icon.

JCheckBox(String s): Creates an initially unselected check box with text.

JCheckBox(String text, boolean selected): Creates a checkbox with text and specifies whether or not it is initially selected.

JCheckBox(Action a): Creates a checkbox where properties are taken from the Action supplied.

Table 13.5: JCheckBox Methods

JCheckBox Methods	Description
AccessibleContext getAccessibleContext()	It is used to get the AccessibleContext associated with this JCheckBox.
protected String paramString()	It returns a string representation of this JCheckBox.

Example:

```

import javax.swing.*;
import java.awt.event.*;
public class CheckBoxExample
{
    CheckBoxExample() {
        JFrame f= new JFrame("CheckBox Example");
        final JLabel label = new JLabel();
        label.setHorizontalAlignment(JLabel.CENTER);
        label.setSize(400,100);
        JCheckBox checkbox1 = new JCheckBox("C++");
        checkbox1.setBounds(150,100, 50,50);
        JCheckBox checkbox2 = new JCheckBox("Java");
        checkbox2.setBounds(150,150, 50,50);
        f.add(checkbox1); f.add(checkbox2); f.add(label);
        checkbox1.addItemListener(new ItemListener() {
            public void itemStateChanged(ItemEvent e) {
                label.setText("C++ Checkbox: "
                    + (e.getStateChange()==1?"checked":"unchecked"));
            }
        });
        checkbox2.addItemListener(new ItemListener() {
            public void itemStateChanged(ItemEvent e) {
                label.setText("Java Checkbox: "
                    + (e.getStateChange()==1?"checked":"unchecked"));
            }
        });
        f.setSize(400,400);
        f.setLayout(null);
        f.setVisible(true);
    }
}

```

```

    }
    public static void main(String args[])
    {
        new CheckBoxExample();
    }
}

```

JRadioButton: used to choose one option from multiple options.

JRadioButton Constructor:

JRadioButton(): Creates an unselected radio button with no text.

JRadioButton(String s): Creates an unselected radio button with specified text.

JRadioButton(String s, boolean selected): Creates a radio button with the specified text and selected status.

Table 13.6: JRadioButton Methods

JRadioButton Methods	Description
void setText(String s)	It is used to set specified text on button.
String getText()	It is used to return the text of the button.
void setEnabled(boolean b)	It is used to enable or disable the button.
void setIcon(Icon b)	It is used to set the specified Icon on the button.
Icon getIcon()	It is used to get the Icon of the button.
void setMnemonic(int a)	It is used to set the mnemonic on the button.
void addActionListener(ActionListener a)	It is used to add the action listener to this object.

Example:

```

import javax.swing.*;
public class RadioButtonExample
{
    JFrame f;
    RadioButtonExample(){
        f=new JFrame();
        JRadioButton r1=new JRadioButton("A) Male");
        JRadioButton r2=new JRadioButton("B) Female");
        r1.setBounds(75,50,100,30);
        r2.setBounds(75,100,100,30);
        ButtonGroup bg=new ButtonGroup();
        bg.add(r1);bg.add(r2);
        f.add(r1);f.add(r2);
        f.setSize(300,300);
        f.setLayout(null);
        f.setVisible(true);
    }
    public static void main(String[] args)

```



```

    {
        new RadioButtonExample();
    }
}

```

JComboBox: used to show popup menu of choices. Choice selected by user is shown on the top of a menu.

JComboBox Constructor:

JComboBox():Creates a JComboBox with a default data model.

JComboBox(Object[] items): Creates a JComboBox that contains the elements in the specified array.

JComboBox(Vector<?> items): Creates a JComboBox that contains the elements in the specified Vector.

Table 13.7: JComboBox Methods

JComboBox Methods	Description
void addItem(Object anObject)	It is used to add an item to the item list.
void removeItem(Object anObject)	It is used to delete an item to the item list.
void removeAllItems()	It is used to remove all the items from the list.
void setEditable(boolean b)	It is used to determine whether the JComboBox is editable.
void addActionListener(ActionListener a)	It is used to add the ActionListener.

Example:

```

import javax.swing.*;
public class ComboBoxExample
{
    JFrame f;
    ComboBoxExample(){
        f=new JFrame("ComboBox Example");
        String country[]={"India","Aus","U.S.A","England","Newzealand"};
        JComboBox cb=new JComboBox(country);
        cb.setBounds(50, 50,90,20);
        f.add(cb);
        f.setLayout(null);
        f.setSize(400,500);
        f.setVisible(true);
    }
    public static void main(String[] args)
    {
        new ComboBoxExample();
    }
}

```

JTable: JTable class is used to display data in tabular form.

JTable Constructors:

JTable():Creates a table with empty cells.

JTable(Object[][] rows, Object[] columns): Creates a table with the specified data.

Example:

```
import javax.swing.*;
public class TableExample
{
    JFrame f;
    TableExample(){
        f=new JFrame();
        String data[][]={ {"101","Amit","670000"},
                           {"102","Jai","780000"},
                           {"101","Sachin","700000"} };
        String column[]={"ID","NAME","SALARY"};
        JTable jt=new JTable(data,column);
        jt.setBounds(30,40,200,300);
        JScrollPane sp=new JScrollPane(jt);
        f.add(sp);
        f.setSize(300,400);
        f.setVisible(true);
    }
    public static void main(String[] args)
    {
        new TableExample();
    }
}
```

JOptionPane: used to provide standard dialog boxes such as message dialog box, confirm dialog box and input dialog box. These dialog boxes are used to display information or get input from the user.

JOptionPane Constructors:

JOptionPane(): It is used to create a JOptionPane with a test message.

JOptionPane(Object message): It is used to create an instance of JOptionPane to display a message.

JOptionPane(Object message, int messageType): It is used to create an instance of JOptionPane to display a message with specified message type and default options.

Table 13.8: JOptionPane Methods

JOptionPane Methods	Description
JDialog createDialog(String title)	It is used to create and return a new parentless JDialog with the specified title.
static void showMessageDialog(Component parentComponent, Object message)	It is used to create an information-message dialog titled "Message".
static void showMessageDialog(Component parentComponent, Object message, String title, int messageType)	It is used to create a message dialog with given title and messageType.
static int showConfirmDialog(Component parentComponent, Object message)	It is used to create a dialog with the options Yes, No and Cancel; with the title, Select an Option.

static String showInputDialog(Component parentComponent, Object message)	It is used to show a question-message dialog requesting input from the user parented to parentComponent.
void setInputValue(Object newValue)	It is used to set the input value that was selected or input by the user.

Example:

```
import javax.swing.*;
import java.awt.event.*;
public class OptionPaneExample extends WindowAdapter
{
    JFrame f;
    OptionPaneExample(){
        f=new JFrame();
        f.addWindowListener(this);
        f.setSize(300, 300);
        f.setLayout(null);
        f.setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
        f.setVisible(true);
    }
    public void windowClosing(WindowEvent e) {
        int a=JOptionPane.showConfirmDialog(f,"Are you sure?");
        if(a==JOptionPane.YES_OPTION){
            f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        }
    }
    public static void main(String[] args)
    {
        new OptionPaneExample();
    }
}
```

JScrollBar: used to add horizontal and vertical scrollbar.

JScrollBar Constructors:

JScrollBar():Creates a vertical scrollbar with the initial values.

JScrollBar(int orientation): Creates a scrollbar with the specified orientation and the initial values.

JScrollBar(int orientation, int value, int extent, int min, int max): Creates a scrollbar with the specified orientation, value, extent, minimum, and maximum.

Example:

```
import javax.swing.*;
class ScrollBarExample
{
    ScrollBarExample(){
        JFrame f= new JFrame("Scrollbar Example");
        JScrollBar s=new JScrollBar();
        s.setBounds(100,100, 50,100);
    }
}
```

```

        f.add(s);
        f.setSize(400,400);
        f.setLayout(null);
        f.setVisible(true);
    }
    public static void main(String args[])
    {
        new ScrollBarExample();
    }
}

```

JSlider: used to create the slider. By using JSlider, a user can select a value from a specific range.

JSlider Constructors:

JSlider(): creates a slider with the initial value of 50 and range of 0 to 100.

JSlider(int orientation): creates a slider with the specified orientation set by either JSlider.HORIZONTAL or JSlider.VERTICAL with the range 0 to 100 and initial value 50.

JSlider(int min, int max): creates a horizontal slider using the given min and max.

JSlider(int min, int max, int value): creates a horizontal slider using the given min, max and value.

JSlider(int orientation, int min, int max, int value): creates a slider using the given orientation, min, max and value.

Table 13.9: JSlider Method

JSlider Method	Description
public void setMinorTickSpacing(int n)	is used to set the minor tick spacing to the slider.
public void setMajorTickSpacing(int n)	is used to set the major tick spacing to the slider.
public void setPaintTicks(boolean b)	is used to determine whether tick marks are painted.
public void setPaintLabels(boolean b)	is used to determine whether labels are painted.
public void setPaintTracks(boolean b)	is used to determine whether track is painted.

Example:

```

import javax.swing.*;
public class SliderExample extends JFrame
{
    public SliderExample() {
        JSlider slider = new JSlider(JSlider.HORIZONTAL, 0, 50, 25);
        slider.setMinorTickSpacing(2);
        slider.setMajorTickSpacing(10);
        slider.setPaintTicks(true);
        slider.setPaintLabels(true);
        JPanel panel=new JPanel();
        panel.add(slider);
        add(panel);
    }
    public static void main(String s[])

```

```

    {
        SliderExample frame=new SliderExample();
        frame.pack();
        frame.setVisible(true);
    }
}

```

11.6 Procedure:

Create a java class, import required Swing packages, create Frame, create Swing components as per requirement and add these components to Swing frame.

11.7 Program and Output:

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class SwingDemo extends JFrame implements ActionListener {
    JTextField tf;
    JLabel label;
    JCheckBox cbJava, cbPython;
    JRadioButton rbMale, rbFemale;
    JButton btnShow;
    JComboBox<String> combo;
    JTable table;

    public SwingDemo() {
        setTitle("Swing Component Demo");
        setLayout(null);
        setSize(500, 500);

        // JLabel
        label = new JLabel("Enter Name:");
        label.setBounds(30, 30, 100, 25);
        add(label);

        // JTextField
        tf = new JTextField();
        tf.setBounds(140, 30, 150, 25);
        add(tf);

        // JCheckBox
        cbJava = new JCheckBox("Java");
        cbJava.setBounds(30, 70, 80, 25);
        cbPython = new JCheckBox("Python");
        cbPython.setBounds(120, 70, 100, 25);
        add(cbJava);
        add(cbPython);
    }
}

```

```

// JRadioButton
rbMale = new JRadioButton("Male");
rbFemale = new JRadioButton("Female");
rbMale.setBounds(30, 110, 80, 25);
rbFemale.setBounds(120, 110, 100, 25);
ButtonGroup bg = new ButtonGroup();
bg.add(rbMale);
bg.add(rbFemale);
add(rbMale);
add(rbFemale);

// JComboBox
String countries[] = {"India", "USA", "UK", "Australia"};
combo = new JComboBox<>(countries);
combo.setBounds(30, 150, 120, 25);
add(combo);

// JTable
String data[][] = {{ "1", "Riya", "IT"}, {"2", "Aarav", "CS"} };
String column[] = {"ID", "Name", "Dept"};
table = new JTable(data, column);
JScrollPane sp = new JScrollPane(table);
sp.setBounds(30, 190, 300, 80);
add(sp);

// JButton
btnShow = new JButton("Show Info");
btnShow.setBounds(30, 290, 120, 30);
btnShow.addActionListener(this);
add(btnShow);

setVisible(true);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}

public void actionPerformed(ActionEvent e) {
    String name = tf.getText();
    String lang = "";
    if (cbJava.isSelected()) lang += "Java ";
    if (cbPython.isSelected()) lang += "Python ";
    String gender = rbMale.isSelected() ? "Male" : "Female";
    String country = (String) combo.getSelectedItem();
    JOptionPane.showMessageDialog(this,
        "Name: " + name + "\nGender: " + gender + "\nLanguages: " + lang + "\nCountry: " +
country);
}

public static void main(String[] args) {
    new SwingDemo();
}

```

```
}  
}
```

```
C:\Users\Shravani\Desktop>javac SwingDemo.java  
  
C:\Users\Shravani\Desktop>java SwingDemo  
  
Launching SwingDemo GUI,..  
SwingDemo window opened.  
  
Button clicked! Showing user information:  
    Name: Riya  
    Gender: Female  
    Languages: Java Python  
    Country: India  
  
C:\Users\Shravani\Desktop>
```

11.8 Conclusion:

This experiment demonstrates how **Swing components** are used to design interactive GUI applications in Java. By combining elements like buttons, labels, text fields, checkboxes, radio buttons, combo boxes, tables, and dialog boxes, developers can create user-friendly interfaces. Swing provides flexibility, event handling, and platform-independent design for modern desktop applications.

11.9 Questions:

1. What is a container in a GUI?

A **container** is a component that holds and organizes other GUI components (like buttons, labels, text fields). Examples include JFrame, JPanel, and JDialog.

2. Name container classes.

Common container classes are:

- JFrame
- JPanel
- JDialog

- JApplet
- JWindow

3. What are the differences between Swing and AWT?

Feature	AWT	Swing
Library	Part of Java's original Abstract Window Toolkit	Part of Java Foundation Classes (JFC)
Platform Dependency	Platform-dependent (uses native OS components)	Platform-independent (pure Java components)
Look and Feel	Fixed, native OS look	Customizable look and feel
Lightweight/Heavyweight	Heavyweight components	Lightweight components
Package	java.awt	javax.swing